Q4. Best-First vs Hill Climbing: Expanding the Search Space and Action Space

> **Question.** What do best-first search and hill climbing have in common? How can best-first search expand the search space? Can the action space and the search space grow together? Using Diagram 3, can we control the convergence speed, and can it differ at A, B and C? For Q3, define two extra operations that expand the search space and describe their effect on the goal state.

1. What they have in common

- Both are **informed (heuristic) searches**. They use an evaluation score to decide which design to look at next.
- Both are **greedy**: they pick what looks best *now*, not what is best after many steps.
- Both expand a node by applying the same legal actions and scoring the children.

	Hill climbing	Best-first search
Memory	Only the current design	An OPEN list of every design generated
Next node	Best neighbour of the current one	Best design anywhere in OPEN
Backtracking	None, so it gets stuck on a local maximum	Yes, it can jump back to an older, promising node
Cost	Cheap	More memory and more designs scored

In short, hill climbing is best-first search with a memory of size one.

2. Assumptions

- Diagram 3 is a score hill f(x) with its peak at x = 6. A is the foot (x = 1.3), B is the slope and C is the peak.
- The airport model is the one from Q3: blueprint (10 gates, 6 counters, area 100, shops landside, waiting area 14).
- Best-first gets a budget of 400 expansions, so the comparison is fair.

3. How can best-first search expand the search space?

Hill climbing throws away every neighbour it does not choose. Best-first keeps them all in OPEN. When the current branch stops improving, it simply pops the next most promising design, even one from several steps ago. So the explored space grows in many directions at once. In our run, best-first scored **875** designs with actions a–e, while hill climbing scored only **117**.

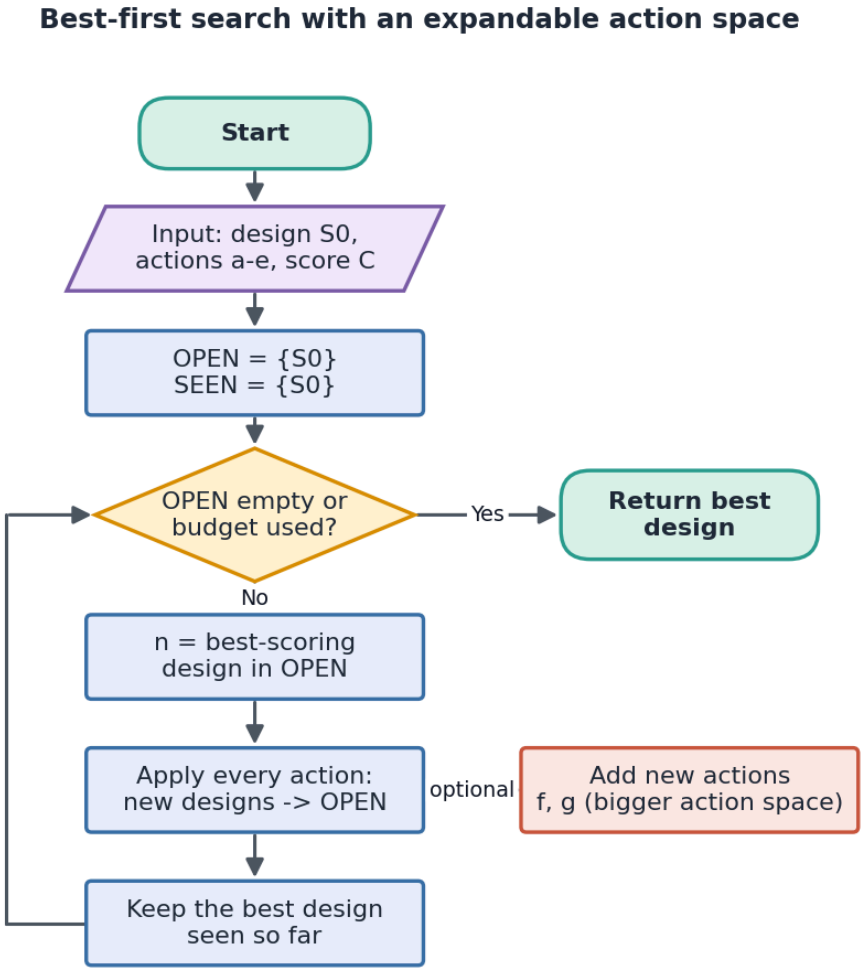


Figure 1. Best-first search. The optional box shows the action space growing.

```
function BestFirst(S0, actions, budget):
    OPEN ← {S0};  SEEN ← {S0};  best ← S0
    while OPEN not empty and expansions < budget:
        n ← design in OPEN with the highest Score
        best ← better of (best, n)
        for each action in actions:                # the action space can be expanded here
            child ← apply(action, n)
            if child is legal and child not in SEEN: add to OPEN and SEEN
    return best
```

4. Can the action space and search space expand at the same time? Yes

Every new action creates new children, so a bigger action space automatically makes the reachable search space bigger. In the airport, **designs reachable in 6 moves rise from 650 (actions a–e) to 1765 (actions a–g)**. That is the idea of "expand search space + expand action space together": a new action is a new edge, and a new edge leads to new states.

5. Controlling convergence speed (Diagram 3)

Yes, we can control it, and it can be different at A, B and C. The tool is the **step size** (in simulated annealing the same job is done by the temperature).

Zone	Where	Rule used	Why
A	Foot, flat ground	Big step (1.2)	The slope is tiny, so small steps would crawl. Be bold.
B	Steep slope	Medium step (0.6)	Follow the gradient steadily. Fast and safe.
C	Near the peak	Halve the step after each overshoot	A big step would jump over the peak. Fine-tune.

Real results from the code (start at A, peak value 85.00):

Strategy	Moves	Final x	Score	Gap to peak
Fixed small step 0.1	47	6.000	85.00	0.000
Fixed big step 1.5	3	5.800	84.38	0.623
Adaptive A / B / C	**12**	**5.997**	**85.00**	**0.000**

The small step is accurate but slow. The big step is fast but misses the peak. The adaptive step gets both: it reached the peak in 12 moves, about 4 times faster than the small step.

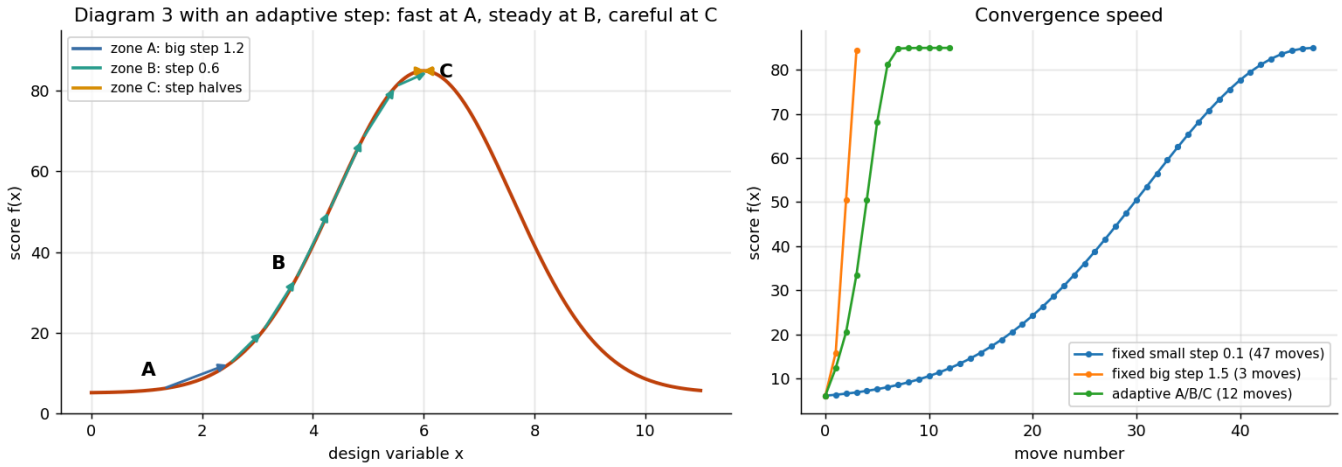


Figure 2. Left: Diagram 3 with the adaptive path (blue = A, green = B, orange = C). Right: score against move number for the three strategies.

6. Two extra operations for the Q3 airport

Op	Definition	Why it expands the search space
f	**Add a new pier:** +20 floor area, +2 gates, +8 seats in one move.	A macro-move. It changes the building footprint, which a–e can never do (they only remove space).
g	**Gate + seats together:** +1 gate and +4 waiting units in one move.	A combined action. It climbs the ridge where gates and seats must grow together.

Impact on the goal state (real run)

Method	Actions	Designs scored	Best score	Best design (gates, counters, area)
Hill climbing	a–e	117	79.40	14, 13, 100
Best-first (400)	a–e	875	79.40	14, 13, 100
Hill climbing	a–g	158	82.49	17, 14, 120
Best-first (400)	**a–g**	**1856**	**82.87**	**19, 14, 128**

- **Search alone is not enough.** With a–e, even best-first stays at 79.40, because every design it can reach is no better.
- **A bigger action space moves the peak itself.** Adding f and g lifts the best score from 79.40 to 82.49 (hill climbing) and 82.87 (best-first). Hill climbing used f once, at move 14.
- The goal state is no longer the same building. It now has 19 gates and a larger floor, so the **goal state moved**.
- The price: about 12 times more designs scored (1856 for best-first vs 158 for hill climbing).

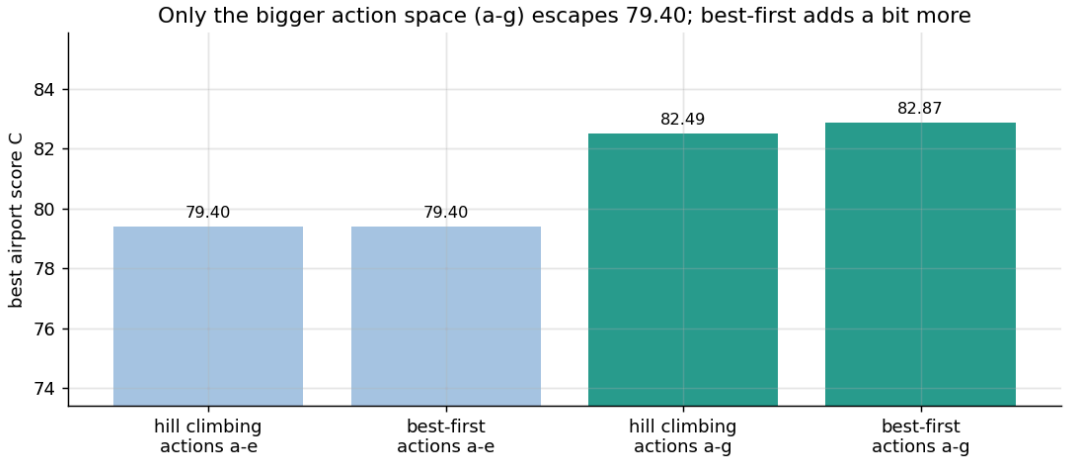


Figure 3. Best score found with each method and action set.

7. Code and real run

File Edit View Run Terminal Help — q04_code.py - Q04_BestFirst_vs_HillClimbing - Python IDE

q04_code.py

```python
def hill_climb(start: tuple, actions: dict) -> tuple:
    current, evaluated, used = start, 1, []
    while True:
        options = [(score(n), name, n) for name, act in actions.items() if legal(n := act(current))]
        evaluated += len(options)
        best_score, name, best = max(options, key=lambda o: o[0])  # first best move wins ties
        if best_score <= score(current):
            return current, evaluated, used
        current = best
        used.append(name)


def best_first(start: tuple, actions: dict, budget: int = 400) -> tuple:
    """Best-first keeps every generated design in OPEN, so it can leave a local peak."""
    tie, best, expanded = 0, start, 0
    frontier, seen = [(-score(start), tie, start)], {start}
    while frontier and expanded < budget:
        _, _, s = heapq.heappop(frontier)  # most promising design anywhere in OPEN
        expanded += 1
        best = max(best, s, key=score)
        for act in actions.values():
            n = act(s)
            if legal(n) and n not in seen:
                seen.add(n)
                tie += 1
                heapq.heappush(frontier, (-score(n), tie, n))
    return best, len(seen)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS C:\Users\USER\questions\Answers\Q04_BestFirst_vs_HillClimbing> python q04_code.py
PART A  Diagram 3 convergence (start A at x = 1.3, peak C at x = 6.00, f = 85.00)
  strategy              | moves | final x | f(x)  | gap to peak
  fixed small step 0.1  |    47 |   6.000 | 85.00 | 0.000
  fixed big step 1.5    |     3 |   5.800 | 84.38 | 0.623
  adaptive A/B/C        |    12 |   5.997 | 85.00 | 0.000
  adaptive step in zone A: 1.2
  adaptive step in zone B: 0.6, 0.6, 0.6, 0.6, 0.6, 0.6
  adaptive step in zone C: 0.15, 0.075, 0.0375, 0.01875, 0.009375
PART B  Q3 airport + f: new pier (+20 floor, +2 gates, +8 seats), g: gate + 4 seats
  designs reachable in 6 moves: a-e = 650, a-g = 1765
  method                | actions | designs scored | best score | design (gates, counters, area)
  hill climbing         | a-e     |            117 |      79.40 | 14, 13, 100
  best-first (400)      | a-e     |            875 |      79.40 | 14, 13, 100
  hill climbing         | a-g     |            158 |      82.49 | 17, 14, 120
  best-first (400)      | a-g     |           1856 |      82.87 | 19, 14, 128
  hill climbing a-g, moves in order: e+ d+ d+ d+ g b b b b g a b b f a a b b
charts saved to images/
PS C:\Users\USER\questions\Answers\Q04_BestFirst_vs_HillClimbing>
```

main — Ln 133, Col 1 — Spaces: 4 — UTF-8 — CRLF — Python 3.12.3

Figure 4. Hill-climbing and best-first functions (lines 107–133) and the real terminal output.

8. Summary

> Best-first and hill climbing are both greedy and heuristic. Best-first adds an OPEN list, so it can leave a local peak and explore more. Step size (or temperature) controls convergence: big at A, steady at B, shrinking at C. The biggest gain came from expanding the *action space* with f and g, which grew the search space from 650 to 1765 designs and moved the best score from 79.40 to 82.87.

Files in this folder: `q04_code.py` (source), `q04_code.ipynb` (Colab/Jupyter version, already run), `q04_output.txt` (console output), `images/` (all figures).